Os nit 4 – Orange Program Assignment

Screenshots

1a

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs
o@zebra:~/PES2UG24CS475-pes-vcs$ ./test_objects
ed blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
ct stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9a
2
: blob storage
: deduplication
: integrity check

Phase 1 tests passed.
o@zebra:~/PES2UG24CS475-pes-vcs$
```

1b

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs
kesho@zebra:~/PES2UG24CS475-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9a
afc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
kesho@zebra:~/PES2UG24CS475-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

2a

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs
kesho@zebra:~/PES2UG24CS475-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

2b

```
kesho@zebra:~/PES2UG24CS475-pes-vcs$ find .pes/objects -type f
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/9a/63be660bd5deae5453cb541c1c95e362717a4532a734d9e2fe0f96713686bd
.pes/objects/0f/611362716d2436bf2fda0824cac1bfba60ec96a5f65a709f4dcd40ddfa5366
.pes/objects/24/15087962f088a95d8b1609626c6e7ee0a7794fe094abf60f10e129eb4ae41d
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
kesho@zebra:~/PES2UG24CS475-pes-vcs$ echo $I am taking the 2nd last path
am taking the 2nd last path
kesho@zebra:~/PES2UG24CS475-pes-vcs$ xxd .pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

3a

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs

kesho@zebra:~/PES2UG24CS475-pes-vcs$ make pes
./pes add file1.txt file2.txt
./pes status
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  commit.c
  untracked:  index.h
  untracked:  test_tree
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  pes.c
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  README.md
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  pes.h

kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

3b

kesho@zebra: ~/PES2UG24CS475-pes-vcs

```
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  commit.c
  untracked:  index.h
  untracked:  test_tree
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  pes.c
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  README.md
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  pes.h

kesho@zebra:~/PES2UG24CS475-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d3376
39025de576db9ebb4 1776346570 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9
f7e65d6f606199f7f 1776346570 6 file2.txt
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

4a

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs
kesho@zebra:~/PES2UG24CS475-pes-vcs$ ./pes log
commit 0f611362716d2436bf2fda0824cac1bfba60ec96a5f65a709f4dcd40ddfa5366
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776349362

    Add farewell

commit 2415087962f088a95d8b1609626c6e7ee0a7794fe094abf60f10e129eb4ae41d
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776349355

    Add world

commit 9a63be660bd5deae5453cb541c1c95e362717a4532a734d9e2fe0f96713686bd
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776349350

    Initial commitA

kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

4b

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs

    Add world

commit 9a63be660bd5deae5453cb541c1c95e362717a4532a734d9e2fe0f96713686bd
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776349350

    Initial commitA

kesho@zebra:~/PES2UG24CS475-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0f/611362716d2436bf2fda0824cac1bfba60ec96a5f65a709f4dcd40ddfa5366
.pes/objects/24/15087962f088a95d8b1609626c6e7ee0a7794fe094abf60f10e129eb4ae41d
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/9a/63be660bd5deae5453cb541c1c95e362717a4532a734d9e2fe0f96713686bd
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

4c

```
kesho@zebra: ~/PES2UG24CS475-pes-vcs
kesho@zebra:~/PES2UG24CS475-pes-vcs$ cat .pes/refs/heads/main
0f611362716d2436bf2fda0824cac1bfba60ec96a5f65a709f4dcd40ddfa5366
kesho@zebra:~/PES2UG24CS475-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

Final test file

```
kesho@zebra:~/PES2UG24CS475-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 53799f2efaf3... Initial commit

Log after first commit:
commit 53799f2efaf322f59e74b57416b084a314113f5c4295917bd9b9489342485ac1
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776350349

    Initial commit◆◆


--- Second Commit ---
Committed: c39e1d34e5cc... Update file.txt

--- Third Commit ---
Committed: 2f4657292847... Add farewell
```

```
--- Full History ---
commit 2f4657292847ddae049080a9e47d59d34cac9b7d2eda51f0202536d95e43da4e
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776350349

    Add farewell

commit c39e1d34e5cc569350527dcb339a04faf7b190d114bad31e60464c6894bda4ab
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776350349

    Update file.txt

commit 53799f2efaf322f59e74b57416b084a314113f5c4295917bd9b9489342485ac1
Author: SHOBHIT RACHIT KESHRI <PES2UG24CS475>
Date:   1776350349

    Initial commitA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
2f4657292847ddae049080a9e47d59d34cac9b7d2eda51f0202536d95e43da4e

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/2f/4657292847ddae049080a9e47d59d34cac9b7d2eda51f0202536d95e43da4e
.pes/objects/53/799f2efaf322f59e74b57416b084a314113f5c4295917bd9b9489342485ac1
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c3/9e1d34e5cc569350527dcb339a04faf7b190d114bad31e60464c6894bda4ab
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
kesho@zebra:~/PES2UG24CS475-pes-vcs$
```

Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answers

5.1

To implement `checkout`, the following steps must occur:

- **File Changes:** The `.pes/HEAD` file must be updated to point to the new branch (e.g., changing from `ref: refs/heads/main` to `ref: refs/heads/feature`).
- **Working Directory:** The system must clear the current working directory and repopulate it with the files described in the **Tree** object associated with the target branch's latest commit.
- **Complexity:** The complexity arises from **safety**. You cannot simply "delete everything and start over." You must handle "dirty" files (uncommitted changes) so the user doesn't lose work. Additionally, the system must efficiently handle directory structures—creating

subfolders only when necessary and removing old ones that no longer exist in the target branch.

5.2

A conflict is detected through a three-way comparison:

1. **Working Dir vs. Index:** Use `stat()` to compare the current file's `mtime` and `size` against the metadata in the `.pes/index`. If they differ, the file is potentially dirty.
2. **Hashing:** If metadata suggests a change, hash the working file. If the new hash differs from the hash stored in the index, the file is definitely **dirty**.
3. **Index vs. Target:** Compare the file's hash in the index with the file's hash in the **Target Commit's Tree**. If the working file is dirty *and* it differs from the version in the branch you are switching to, the checkout must be refused to prevent overwriting uncommitted work.

Q5.3: What happens if you make commits in a "Detached HEAD" state? How could a user recover those commits?

5.3

In a **Detached HEAD** state, the `.pes/HEAD` file contains a raw commit hash instead of a branch reference.

- **New Commits:** If you commit, the new commit's `parent` will be the current hash in `HEAD`. The `HEAD` will then update to the new commit's hash. However, since no branch (like `main`) points to this new commit, it is "orphaned."
- **Recovery:** If the user switches away, they lose the pointer to that commit. They can recover it by finding the hash in their terminal history (from when they committed) and creating a new branch at that hash: `pes branch recovery-branch <orphaned-hash>`. In real Git, one would use the `reflog`.

6.1

The standard algorithm is **Mark-and-Sweep**:

1. **Mark Phase:** Start at all **Roots** (all files in refs/heads/, the index, and HEAD). Use a **Hash Set** to track "reachable" hashes. Recursively visit every commit, then every tree within those commits, then every blob. For every hash found, add it to the Hash Set.
2. **Sweep Phase:** Iterate through every file in the .pes/objects/ directory. If a file's name (its hash) is **not** in the Hash Set, delete it.
- **Estimation:** In a repo with 100,000 commits and 50 branches, you don't necessarily visit $100,000 \times \text{files}$ objects because of deduplication. However, you must visit

each commit at least once. If each commit has a unique tree, you'd visit **at least 200,000 to 500,000 objects** to build a complete reachability map.

Q6.2: Why is it dangerous to run GC concurrently with a commit? Describe the race condition and Git's solution.

6.2

1. **The Race:** A user runs `pes commit`. The program writes a new **Blob A** to the object store.
2. **The GC:** Simultaneously, `pes gc` runs. It checks the current branches and index. Since the `commit` hasn't finished yet, no branch or index points to **Blob A**.
3. **The Conflict:** The GC identifies **Blob A** as unreachable and deletes it.
4. **The Failure:** The `commit` finishes by writing a Tree that references **Blob A**. You now have a corrupted commit pointing to a file that doesn't exist.

- **Git's Solution:** Git uses a **grace period**. By default, `git gc` will not delete any "loose" object (unreferenced) unless its file modification time (`mtime`) is older than **2 weeks**. This ensures that objects currently being created by active processes are safe from deletion.